

Chapter 3: Data Representation

NUMBER SYSTEMS CS 2014 2	CONVERTING NUMBERS TO DECIMAL CS 2014 4	CONVERTING NUMBERS TO BINARY CS 2014 8	CONVERTING BETWEEN NUMBERS SYSTEMS CS 2014 14
CODING SYSTEMS CS 2014 23	COMPLEMENTS CS 2014 27	<u>FIXED</u> POINT REPRESENTATION OF <u>SIGNED</u> NUMBERS CS 2014 29	FLOATING POINT REPRESENTATION CS 2014 32
FLOATING POINT ARITHMETIC CS 2014 40	ERROR DETECTION CS 2014 44		

NUMBER SYSTEMS

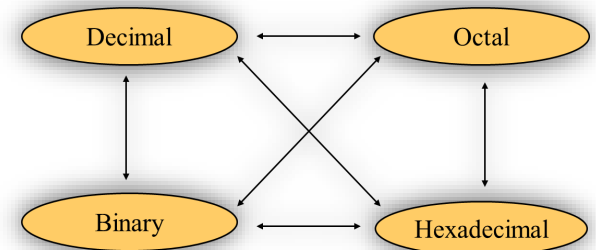
Numbers in Different Bases

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10

Number Systems

Decimal: 0 - 9
Binary: 0 - 1
Octal: 0 - 7
Hexadecimal: 0-9, A, B, C, D, E, F

Conversion among bases



CONVERTING NUMBERS TO DECIMAL

Converting Binary to Decimal

Technique

- Multiply each bit by 2^n , where n is the “weight” of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

Example

$101011_2 \Rightarrow$

1	x	2^0	=	1
1	x	2^1	=	2
0	x	2^2	=	0
1	x	2^3	=	8
0	x	2^4	=	0
1	x	2^5	=	32
				43_{10}

Bit “0”

Converting Octal to Decimal

Technique

- Multiply each bit by 8^n , where n is the “weight” of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

Example

$$\begin{array}{rcll} 724_8 & => & 4 \times 8^0 & = & 4 \\ & & 2 \times 8^1 & = & 16 \\ & & 7 \times 8^2 & = & 448 \\ & & & & \hline & & & & 468_{10} \end{array}$$

Converting Hexadecimal to Decimal

Technique

- Multiply each bit by 16^n , where n is the “weight” of the bit
- The weight is the position of the bit, starting from 0 on the right
- Add the results

Example

$$\begin{array}{rcllcll} \text{ABC}_{16} \Rightarrow & \text{C} & \times & 16^0 & = & 12 & \times & 1 & = & 12 \\ & \text{B} & \times & 16^1 & = & 11 & \times & 16 & = & 176 \\ & \text{A} & \times & 16^2 & = & 10 & \times & 256 & = & 2560 \\ & & & & & & & & & \hline & & & & & & & & & 2748_{10} \end{array}$$

CONVERTING NUMBERS TO BINARY

Converting Fixed Point Decimal to Binary

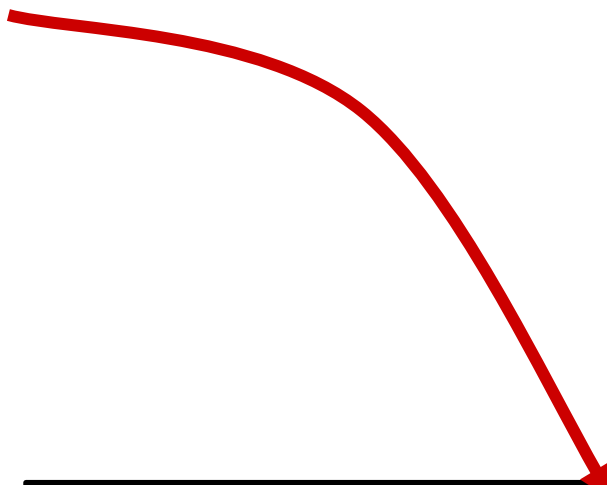
Technique

- Divide by two, keep track of the remainder
- First remainder is bit 0 (LSB, least-significant bit)
- Second remainder is bit 1
- Etc.

Example

$$125_{10} = ?_2$$

2		125	
2		62	1
2		31	0
2		15	1
2		7	1
2		3	1
2		1	1
		0	1



$125_{10} = 1111101_2$

Converting Floating Point Decimal to Binary

Technique

- Integer part: Divide by two, keep track of the remainder
- Fractional part: Multiply by two and keep track of the integer part of the result

Example

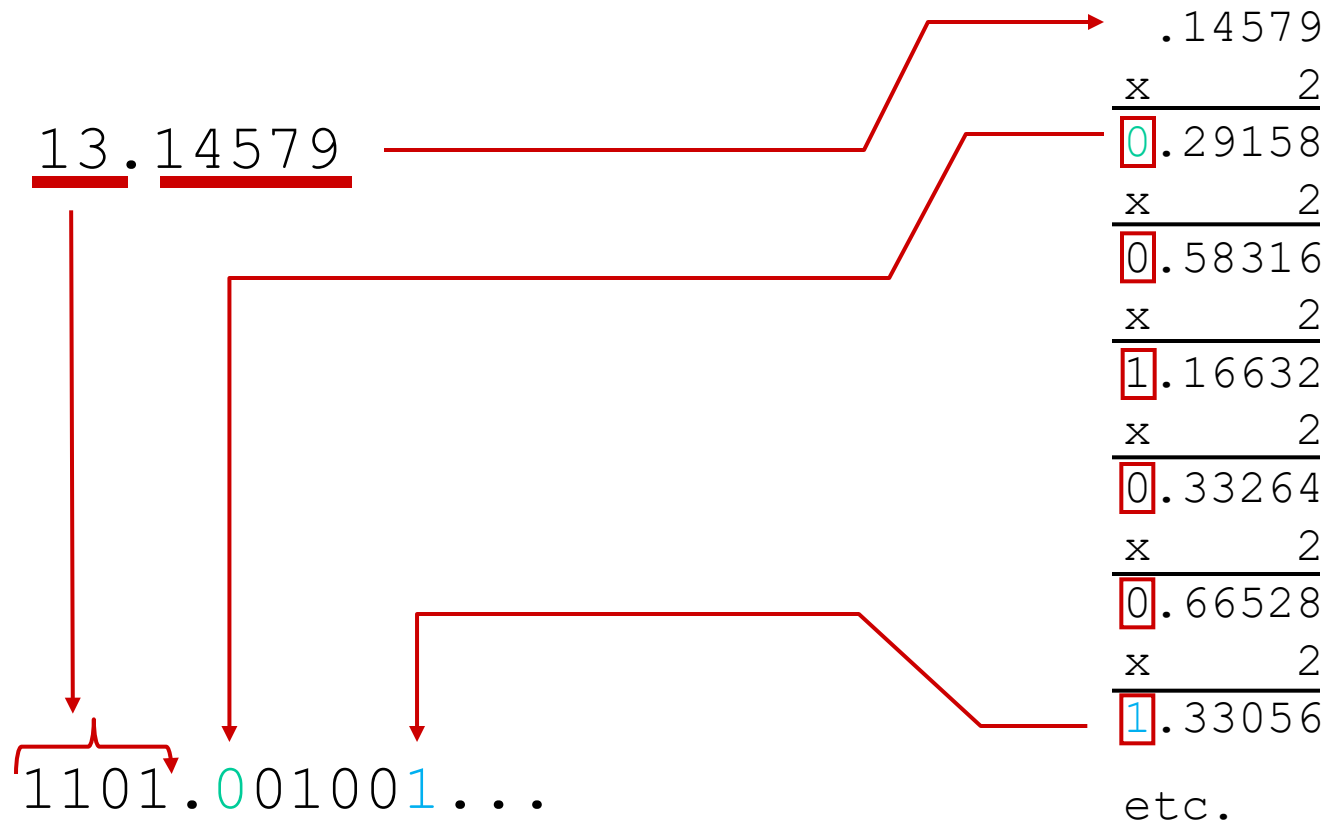
$$\begin{array}{r|l} 41 & \\ 20 & 1 \\ 10 & 0 \\ 5 & 0 \\ 2 & 1 \\ 1 & 0 \\ 0 & \textcolor{red}{1} \end{array} \quad \longrightarrow \quad \begin{array}{l} \text{binary} \\ 0.6875 \\ \quad \times 2 \\ \hline \textcolor{brown}{1}.3750 \\ \quad \times 2 \\ \hline 0.7500 \\ \quad \times 2 \\ \hline 1.5000 \\ \quad \times 2 \\ \hline 1.0000 \end{array}$$

$(41)_{10} = (\textcolor{red}{1}01001)_2$

$(0.6875)_{10} = (0.\textcolor{brown}{1}011)_2$

$(41.6875)_{10} = (\textcolor{green}{1}01001.\textcolor{brown}{1}011)_2$

Converting Floating Point Decimal to Binary (2nd example)



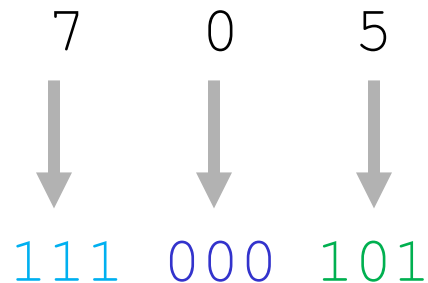
Converting Octal to Binary

Technique

- Convert each **octal** digit to a 3-bit equivalent binary representation

Example

$$705_8 = ?_2$$



Binary (Base 2)	Octal (Base 8)
000	00
001	01
010	02
011	03
100	04
101	05
110	06
111	07

$$705_8 = 111000101_2$$

Converting Hexadecimal to Binary

Technique

- Convert each **hexadecimal** digit to a 4-bit equivalent binary representation

Example

$$10AF_{16} = ?_2$$

1 0 A F

↓ ↓ ↓ ↓

0001 0000 1010 1111

$$10AF_{16} = 0001000010101111_2$$

Numbers in Different Bases

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
00	0000	00	00
01	0001	01	01
02	0010	02	02
03	0011	03	03
04	0100	04	04
05	0101	05	05
06	0110	06	06
07	0111	07	07
08	1000	10	08
09	1001	11	09
10	1010	12	0A
11	1011	13	0B
12	1100	14	0C
13	1101	15	0D
14	1110	16	0E
15	1111	17	0F
16	10000	20	10

Number Systems

Decimal: 0 - 9
Binary: 0 - 1
Octal: 0 - 7
Hexadecimal: 0 - 9, A, B, C, D, E, F

Conversion among bases

CS 2614

CONVERTING BETWEEN NUMBERS SYSTEMS

Converting Decimal to Octal

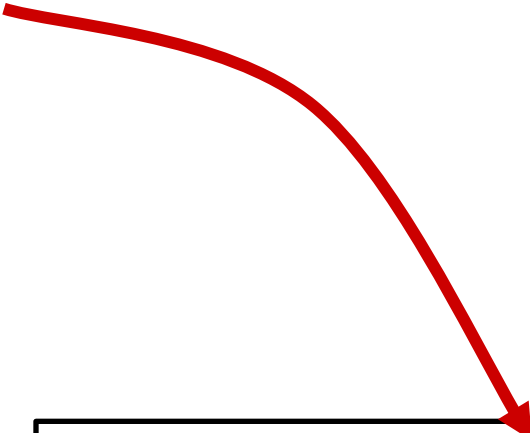
Technique

- Divide by 8
- Keep track of the remainder

Example

$$1234_{10} = ?_8$$

8		1234	
8		154	2
8		19	2
8		2	3
		0	2


$$1234_{10} = 2322_8$$

Converting Decimal to Hexadecimal

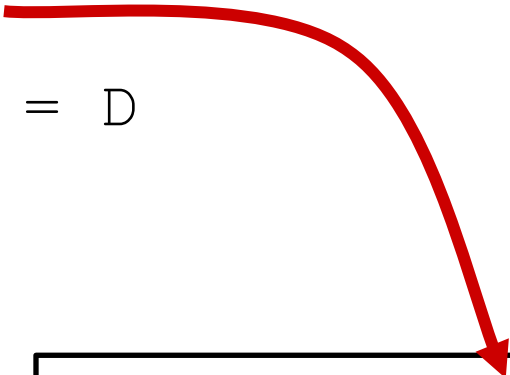
Technique

- Divide by **16**
- Keep track of the remainder

Example

$$1234_{10} = ?_{16}$$

16		1234	
16		77	2
16		4	13 = D
		0	4


$$1234_{10} = 4D2_{16}$$

Converting Binary to Octal

Technique

- Group bits in threes, starting on right
- Convert to octal digits

Example

$$1011010111_2 = ?_8$$



$$1011010111_2 = 1327_8$$

Numbers in Different Bases

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

CS 2014

Number Systems

Decimal: 0-9
Binary: 0-1
Octal: 0-7
Hexadecimal: 0-9, A, B, C, D, E, F

Conversion among bases



Converting Binary to Hexadecimal

Technique

- Group bits in fours, starting on right
- Convert to hexadecimal digits

Example

$$1010111011_2 = ?_{16}$$

$$\begin{array}{ccc} 10 & 1011 & 1011 \\ \downarrow & \downarrow & \downarrow \\ 2 & B & B \end{array}$$

$$1010111011_2 = 2BB_{16}$$

Numbers in Different Bases

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
00	0000	00	00
01	0001	01	01
02	0010	02	02
03	0011	03	03
04	0100	04	04
05	0101	05	05
06	0110	06	06
07	0111	07	07
08	1000	10	08
09	1001	11	09
10	1010	12	0A
11	1011	13	0B
12	1100	14	0C
13	1101	15	0D
14	1110	16	0E
15	1111	17	0F
16	10000	20	10

CS 2014



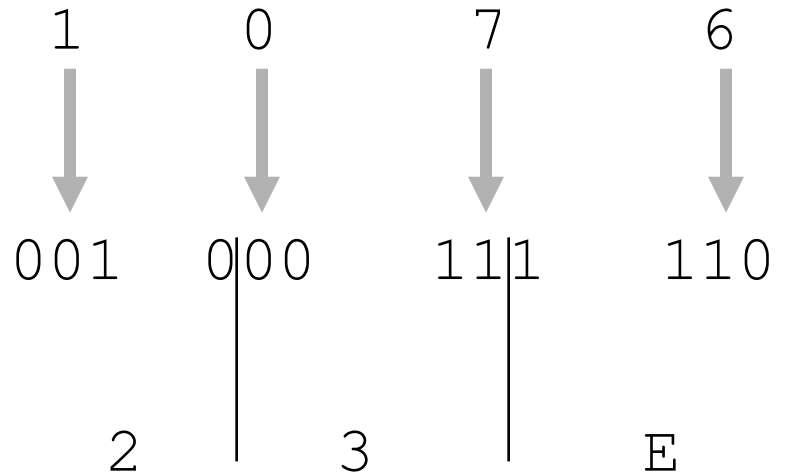
Converting Octal to Hexadecimal

Technique

- Use binary as an intermediary

Example

$$1076_8 = ?_{16}$$



$$1076_8 = 23E_{16}$$

Numbers in Different Bases

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
00	0000	00	00
01	0001	01	01
02	0010	02	02
03	0011	03	03
04	0100	04	04
05	0101	05	05
06	0110	06	06
07	0111	07	07
08	1000	10	08
09	1001	11	09
10	1010	12	0A
11	1011	13	0B
12	1100	14	0C
13	1101	15	0D
14	1110	16	0E
15	1111	17	0F
16	10000	20	10

Number Systems

Decimal: 0 - 9
Binary: 0 - 1
Octal: 0 - 7
Hexadecimal: 0 - 9, A, B, C, D, E, F

Conversion among bases

CS 2614

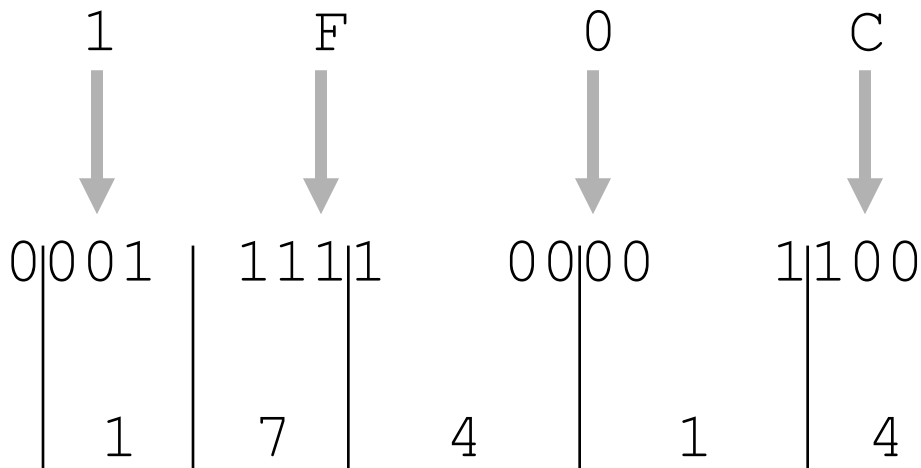
Converting Hexadecimal to Octal

Technique

- Use binary as an intermediary

Example

$$1F0C_{16} = ?_8$$



$$1F0C_{16} = 17414_8$$

Numbers in Different Bases

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)	Hexadecimal (Base 16)
00	0000	00	00
01	0001	01	01
02	0010	02	02
03	0011	03	03
04	0100	04	04
05	0101	05	05
06	0110	06	06
07	0111	07	07
08	1000	10	08
09	1001	11	09
10	1010	12	0A
11	1011	13	0B
12	1100	14	0C
13	1101	15	0D
14	1110	16	0E
15	1111	17	0F
16	10000	20	10

Number Systems

Decimal: 0-9
Binary: 0-1
Octal: 0-7
Hexadecimal: 0-9, A, B, C, D, E, F

Conversion among bases

CS 2614

Exercise: Convert

Decimal	Binary	Octal	Hexa- decimal
33			
	1110101		
		703	
			1AF

Don't use a calculator!

Answer

Decimal	Binary	Octal	Hexa- decimal
33	100001	41	21
117	1110101	165	75
451	111000011	703	1C3
431	110101111	657	1AF

CODING SYSTEMS

BCD

Each **decimal** digit is represented by a 4-bit **binary** number.

Decimal number	Binary-coded decimal (BCD) number
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	0001 0000
20	0010 0000
50	0101 0000
99	1001 1001
248	0010 0100 1000

↑
Code
For one
decimal
digit
↓

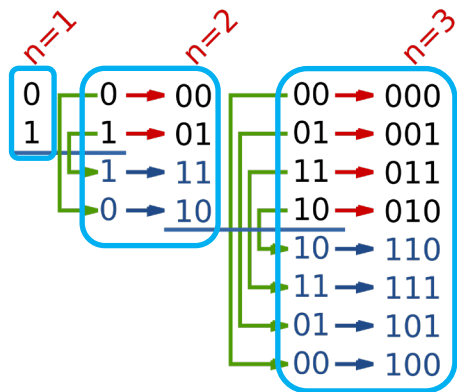
ASCII - uses 7 (or 8) bits to code 128 (or 256) characters

Character	Binary code	Character	Binary code
A	100 0001	U	101 0101
B	100 0010	V	101 0110
C	100 0011	W	101 0111
D	100 0100	X	101 1000
E	100 0101	Y	101 1001
F	100 0110	Z	101 1010

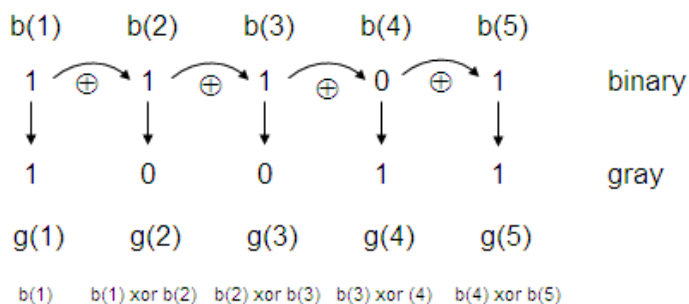
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	

$$T \quad 101 \ 0100 \quad + \quad 010 \ 1011$$

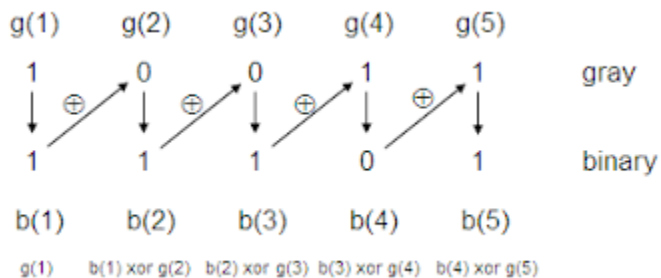
Gray Code



Binary to Gray



Gray to Binary



Gray code	Decimal equivalent	Gray code	Decimal equivalent
0000	0	1100	8
0001	1	1101	9
0011	2	1111	10
0010	3	1110	11
0110	4	1010	12
0111	5	1011	13
0101	6	1001	14
0100	7	1000	15

- Also called reflected binary code
- Only 1 bit changes between adjacent codes

COMPLEMENTS

Complements

Two types of Complements for base-r numbers: r's complement and (r-1)'s complement.

1. To compute (r-1)'s complement, subtract each digit of the number from r-1

$(\text{Number})_r$	$(r-1)$'s complement
$(57)_{10}$	$(42)_{10}$
$(34)_8$	$(43)_8$
$(11010)_2$	$(00101)_2$

99
- 57

42

77
- 34

43

1111 1
- 11010

00101

2. r's complement is the (r-1)'s +1.

Do you know the "trick" for r = 2?

$11010 \rightarrow 00110$

FIXED POINT REPRESENTATION OF SIGNED NUMBERS

Signed Fixed Point Representation

Most significant bit represents sign:
0 = positive number
1 = negative number

2's complement representation can be used for addition of signed numbers.

Important: Must assume a fixed specified number of bits.

+6	0000 0110
<u>+13</u>	<u>0000 1101</u>
+19	0001 0011

-6	1111 1010
<u>+13</u>	<u>0000 1101</u>
+7	0000 0111

+6	0000 0110
<u>-13</u>	<u>1111 0011</u>
-7	1111 1001

-6	1111 1010
<u>-13</u>	<u>1111 0011</u>
-19	1110 1101

Overflow: In general, if the sum of 2 n-bit numbers requires n+1 bits, then an overflow has occurred.

For 2's complement arithmetic, if the XOR of the "carry-in" and "carry-out" of the sign bit is 1, then an overflow exists.

$$\begin{array}{r}
 \text{+70} \quad 0100 \ 0110 \\
 \text{+80} \quad 0101 \ 0000 \\
 \hline
 \text{-106} \quad 1001 \ 0110
 \end{array}$$

Diagram showing carry-in (0) and carry-out (1) for the sign bit.

$$\begin{aligned}
 V &= C_{\text{in}} \oplus C_{\text{out}} \\
 &= 1 \oplus 0 \\
 &= 1 \Rightarrow \text{Overflow!}
 \end{aligned}$$

$$\begin{array}{r}
 \text{+ -1} \quad 1111 \ 1111 \\
 \text{+ -1} \quad 1111 \ 1111 \\
 \hline
 \text{-2} \quad 1111 \ 1110
 \end{array}$$

Diagram showing carry-in (1) and carry-out (1) for the sign bit.

$$\begin{aligned}
 V &= C_{\text{in}} \oplus C_{\text{out}} \\
 &= 1 \oplus 1 \\
 &= 0 \Rightarrow \text{No Overflow!}
 \end{aligned}$$

FLOATING POINT REPRESENTATION

Floating Point Representation

(Textbook is confusing here, please refer to the following notes)

E, the exponent field, is an encoding of a signed binary integer.

The encoded exponent value is biased by 127 (called "**Excess-127**" encoding).

So the encoded numerical value of the exponent is $E - 127$, where E is unsigned.

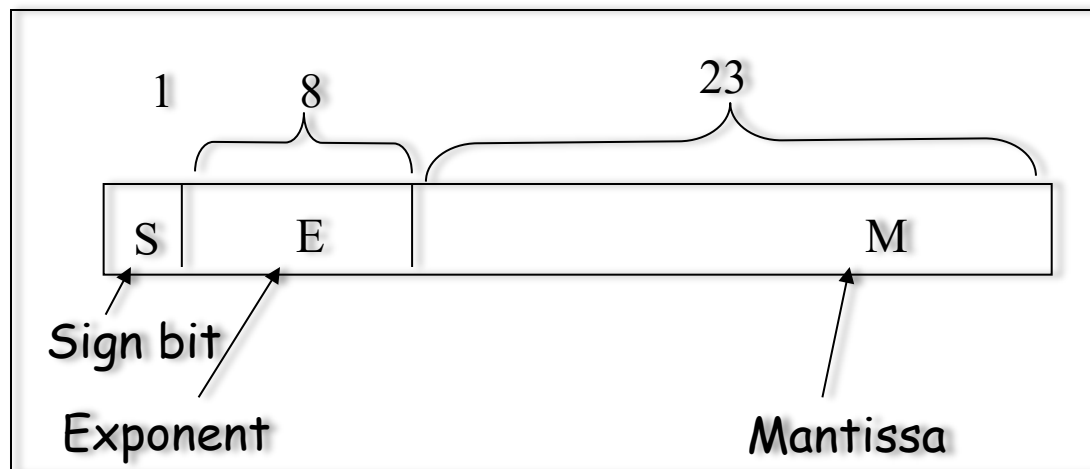
Ex: $E = 0000\ 0001 \Rightarrow EV = -126$

⋮

$E = 0111\ 1111 \Rightarrow EV = 0$

$E = 1000\ 0000 \Rightarrow EV = 1$

IEEE 754 Standard Floating Point Format



Note: $E = 0000\ 0000$ is **reserved** for exact 0

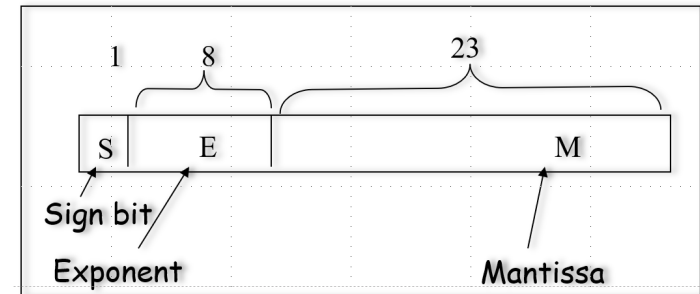
$E = 1111\ 1111$ is reserved for ∞ numbers.

Thus, **range** for EV:

-126 to 127



The Mantissa



Mantissa: This is a signed magnitude # (NOT 2's complement) where S is the sign bit.

The magnitude is normalized to the form **1.bbb...**
and .bbb... (the fraction) is stored in the 23 bits of M.

Note: Because the mantissa is a binary #, the **MSB is always a 1** (for encodings of non-zero numbers).
So the MSB is assumed, but not stored (it is called the phantom bit).

Therefore: The complete signed-magnitude significand effectively has 25 bits, but it is reduced to 24 bits in storage (and unstored phantom bit).

In summary: The numerical value of the complete mantissa is $(-1)^S \times 1.M$
where 1.M is binary.

Examples:

a) $N = 7.5$

$$N = (-1)^S 2^{E-127} (1.M)$$

$$N = 111.1_2 = 1.111 \times 2^2$$

$$\therefore E - 127 = 2$$

$$E = 129, \quad M = 1110 \dots 0$$

0	1000 0001	111 0000 0000 0000 0000 0000
---	-----------	------------------------------

b) $N = 1.5 = 1.1_2 = (1.1)2^0$

$$N = (-1)^S 2^{E-127} (1.M)$$

$$\rightarrow E - 127 = 0$$

$$E = 127, \quad M = 10 \dots 0$$

0	0111 1111	100 0000 0000 0000 0000 0000
---	-----------	------------------------------

c) $N = 1.38$

$2 \times .38 = .76$	0
$2 \times .76 = 1.52$	1
$2 \times .52 = 1.04$	1
$2 \times .04 = .08$	0
$\vdots$	

}

$N=1.38=1.01100001010001111010$

So, $1.38 = (1.38) 2^0 \rightarrow E-127=0$

$E=127$

0	0111 1111	011 0000 1010 0011 1101 0010
---	-----------	------------------------------

d) $N = 9 = 1001.0 = 1.001 \times 2^3 = (-1)^S 2^{E-127} (1.M)$

$So, E - 127 = 3$

$E = 130, \quad M = 0010 \dots 0$

0	1000 0010	001 0000 0000 0000 0000 0000
---	-----------	------------------------------

e) -0

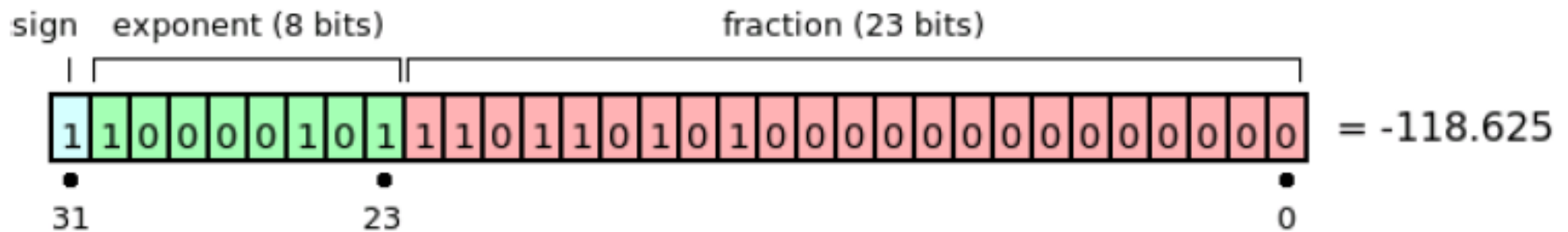
1	0000 0000	000 0000 0000 0000 0000 0000
---	-----------	------------------------------

f) -1

1	0111 1111	000 0000 0000 0000 0000 0000
---	-----------	------------------------------

$$\begin{aligned} N &= (-1)^S 2^{E-127} (1.0) \\ &= (-1)^1 2^0 (1.0) \quad \rightarrow \text{Note: } E - 127 = 0 \end{aligned}$$

A more complex example



Bit values for the IEEE 754 32bit float -118.625

Let us encode the decimal number -118.625 using the IEEE 754 system.

1. First we need to get the sign, the exponent and the fraction. Because it is a negative number, the sign is "1".
2. Now, we write the number (without the sign; i.e. unsigned, no two's complement) using [binary notation](#). The result is 1110110.101. We get the 101 after the decimal like this:
 1. $0.625 \times 2 = 1.25$ which means we write 1 after decimal and move on
 2. $0.25 \times 2 = 0.5$ which means we write 0 after the decimal and move on
 3. $0.5 \times 2 = 1.00$ which means we write 1 after the decimal and we are also finished since we have no residuum left to work with
3. Next, let's move the radix point left, leaving only a 1 at its left: $1110110.101 = 1.110110101 \times 2^6$. This is a normalized floating point number. The first 1 binary digit is dropped. The fraction is the part at the right of the radix point, filled with 0 on the right until we get all 23 bits. That is 11011010100000000000000.
4. The exponent is 6, but we need to convert it to binary and bias it (so the most negative exponent is 0, and all exponents are non-negative binary numbers). For the 32-bit IEEE 754 format, the bias is 127 and so $6 + 127 = 133$. In binary, this is written as 10000101.

Range of Floating-Point numbers

The value of a floating point number N is given by:

$$N = (-1)^S \ 1.M \times 2^{E-127}$$

What is the dynamic range of values N can take?

Small (non-zero) number:

$$E = 0000 \ 0001$$

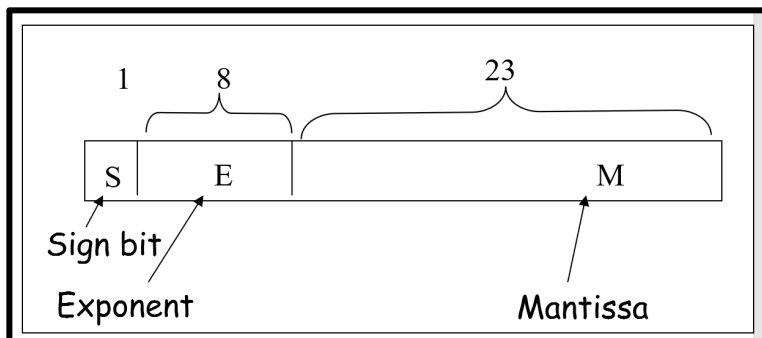
$$M = 000\dots 00$$

$$\text{So, } N = (-1)^0 2^{1-127} 1.0 = (2^{-126}) = 1.18 \times 10^{-38}$$

Large number: $E = 1111 \ 1110 = 254$

$$M = 111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111$$

$$\text{So, } N = (-1)^0 2^{254-127} (1.M) \\ \cong 3.4 \times 10^{38}$$

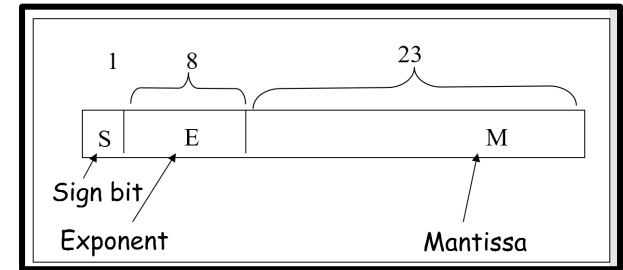


FLOATING POINT ARITHMETIC

Adding two floating point numbers

$$N3 = N1 + N2$$

$$\begin{aligned} &2 \times 10^3 + 3.6 \times 10^4 \\ &= 0.2 \times 10^4 + 3.6 \times 10^4 && \text{(change to the same indices)} \\ &= (0.2 + 3.6) \times 10^4 && \text{(add the numbers)} \\ &= 3.8 \times 10^4 && \text{(answer in scientific notation)} \end{aligned}$$



Basic procedure

1. Determine which number has smallest exponent.
(Suppose it is E_2)
2. Equalize the exponents of N_1 and N_2
(right-shift M_2 , $E_1 - E_2$ places).
3. Add M_1 and shifted M_2 to obtain M_3 .
4. Check for normalization
(not normalized $\Rightarrow$ right-shift M_3 one place and add one to exponent of E_1 to get E_3).

Example

$$\left. \begin{array}{l} N_1 = 7.5 \\ N_2 = 1.5 \end{array} \right\} \begin{array}{l} \text{how to compute} \\ N_3 = N_1 + N_2 \end{array}$$

$N_1 = \overset{S_1}{\underbrace{0}_{\tilde{0}}}$	$\overset{E_1}{\underbrace{1000 \ 0001}}$	$\overset{M_1}{\underbrace{11100 \dots 0}}$
$N_2 = \underset{S_2}{\underbrace{0}}$	$\underset{E_2}{\underbrace{0111 \ 1111}}$	$\underset{M_2}{\underbrace{\text{100} \dots 0}}$

- $E_1 - E_2 = 0000 \ 0010$
 $\Rightarrow E_2$ is smaller than E_1 by 2.
- Right shift M_2 by 2.
 M_2 (shifted 2 $\rightarrow$) = 0**1100** ... 0
- $M_3 = \text{1.11100} + 0.0\text{1100} = \text{10.010} \dots 0$
- right-shift $M_3 \rightarrow \text{1.00100} \dots 0$
 $E_3 = E_1 + 1 = 1000 \ 0010 \Rightarrow N_3 = \underbrace{0}_{S_3} \underbrace{1000 \ 0010}_{E_3} \underbrace{0010 \ \dots \ 0}_{M_3} = +9$

Don't forget the **phantom** bit!

ERROR DETECTION

Error Detection Code - Parity

Message xyz	P(odd)
000	1
001	0
010	0
011	1
100	0
101	1
110	1
111	0

Checkout the following videos:

1. <https://www.youtube.com/watch?v=X8jsijhllIA> between time frames 1m 0s -1m 24s and 5.50 – 7.55
2. https://www.youtube.com/watch?v=jLuj62Gq-1I&ab_channel=GCSEGURU

- Odd number of bit errors can be detected

Error detection with odd parity bit

